

Clinical Layer — Developer Guide

What changed, what was already broken, and what still needs a decision. Every claim here was verified by running it on a clean clone.

FILES CHANGED	TESTS	BEFORE	NEW MODULES	NODE
42	250 passing	43 collection errors	11 (~2,300 lines)	24-alpine

Verification

Not asserted — executed. On a fresh `git clone` with the patch applied:

```
$ git apply CLINICAL-LAYER.patch
$ cd api && python -m pytest tests/ --ignore=tests/integration -q
134 passed

$ python -c "import main; print(len(main.app.routes))"
30

$ python scripts/review_claim_cues.py --measure
precision 1.00  recall 1.00
```

VERIFIED LIVE The PubMed filter, against the real index

Run through a browser on a machine with NCBI access, using the term strings generated by the shipped `build_term()`:

```
(metformin prediabetes) 845 hits
AND humans[MeSH Terms] 610 hits
AND (randomized controlled trial[pt] OR meta-analysis[pt]
OR systematic review[pt] OR practice guideline[pt]) 177 hits
```

Monotonic narrowing, non-zero at every step — exactly what the canary checks. But a smaller number only proves the query changed something. The decisive test reads the MeSH headings off each returned record, on a deliberately animal-heavy topic:

```
unfiltered 20 records – 2 lack the Humans descriptor, 17 tagged Animals/Mice/Rats
filtered 20 records – 0 lack the Humans descriptor
```

`_parse_articles()` was then run on real efetch XML (PMID 32995892) and extracted PMID, title, journal `J Clin Endocrinol Metab`, year 2021, authors, DOI, the `Randomized Controlled Trial` publication type and a 1,733-character structured abstract. A nonsense query returned zero hits without erroring, so genuine absence stays distinguishable from failure.

Same method, same day. Counts first, on (metformin prediabetes) AND humans[MeSH Terms] :

no date window	610 hits	
mindate 1900/01/01	610	← the parameter is accepted; full range == no window
mindate 2000/01/01	605	
maxdate 1999/12/31	5	← 605 + 5 = 610, an exact partition
mindate 2020/01/01	299	

Then the test that actually matters, on digoxin heart failure — chosen because it has substantial 1990s literature — reading the publication year off each of 20 returned records:

no window	20 records – oldest 1993 – 5 of 20 published before 2000
from 2000	20 records – oldest 2000 – 0 of 20 published before 2000

The window does the semantic job, not just an arithmetic one.

The same run caught a defect in our own parser, which is why the figures above are the second set. Reading the publication year the naive way — the first <Year> in the record — **disagreed with the real publication date on 7 of those 20 records**. See the audit below; it is the second finding.

WHAT THAT CHANGED IN THE CODE

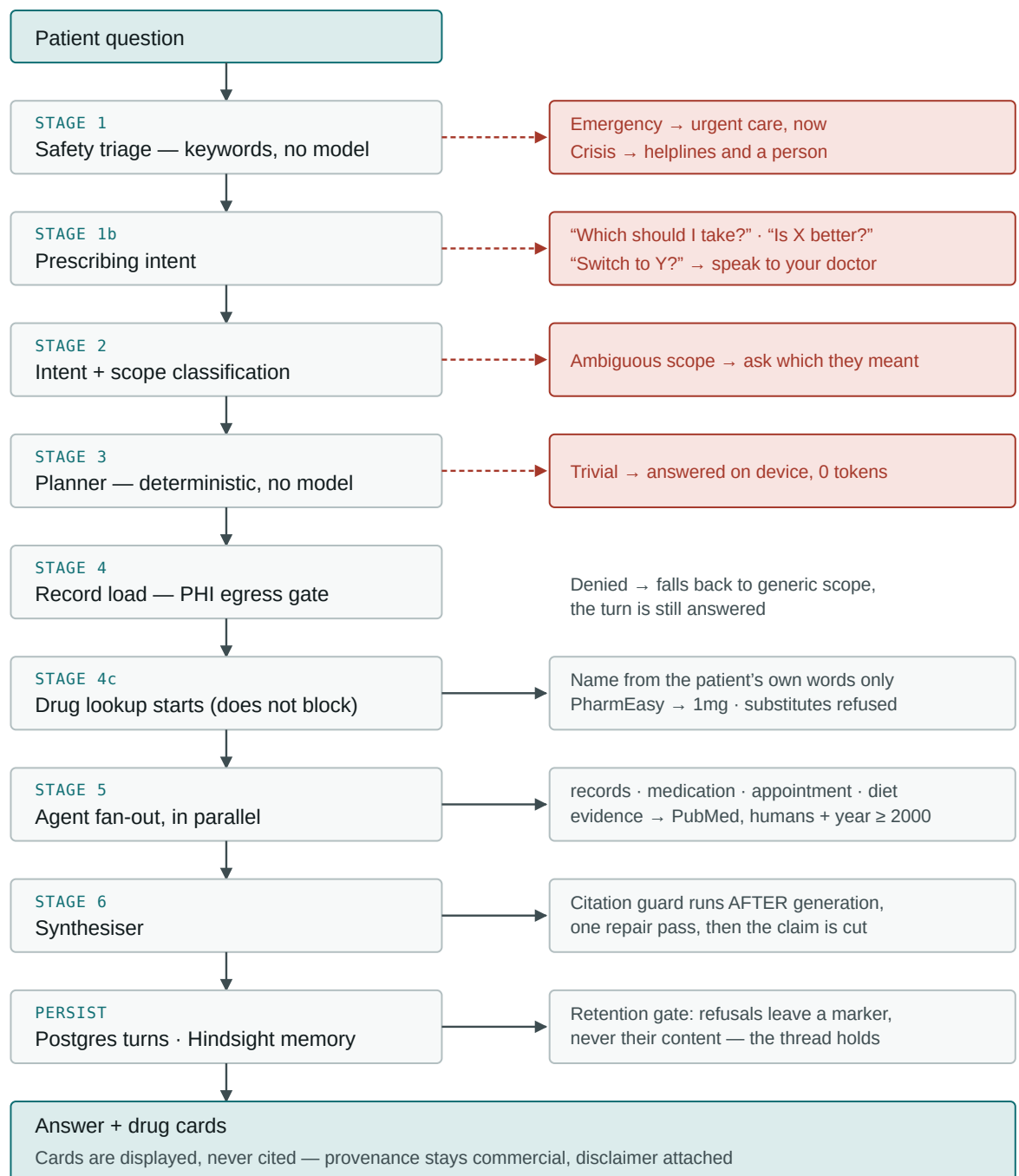
Verifying it exposed a better design. Provenance was assigned from the *query* — "this came back under humans[MeSH Terms], so it is human research". It is now read from each *record*: PubMedArticle.mesh_terms is extracted and is_human_study gates the class.

So if the filter ever silently broke, an animal study returned under a human-only query drops to peer-reviewed-other and the citation guard refuses it as support for a clinical claim. The whole failure mode goes from **failing quiet** to **failing safe**. Two tests cover it, including the downgrade.

One thing still could not be verified here: the integration tests need live Postgres — and the resolver API they import does not exist yet, so a database alone would not help.

How one turn actually flows

Read top to bottom. Every branch to the right is an **exit** — the turn ends there and never reaches the model. That ordering is the whole design: a refusal that depends on an AI choosing to refuse is not a refusal.



Stages 1, 1b, 2b and 3 exit before any model call. Stage 4c overlaps 5 and 6, so its two requests cost no extra wall-clock.

Architecture: three routes, three kinds of evidence

The routing question is not "which tool" but **what may this source be used to say**.
Provenance, not the tool, is what the citation guard enforces.

Clinical question “Does metformin help prediabetes?” PUBMED E-UTILITIES humans[MeSH Terms] published 2000 → now abstract required peer-reviewed-human may support a claim	Named medicine “What is Glycomet 500 used for?” PHARMEASY → IMG name from the patient substitutes refused strength must match commercial shown, never cited	The patient's own data “What was my last HbA1c?” POSTGRES + HINDSIGHT PHI egress gate first retrieved slice, not a dump consent basis recorded patient-record personal, never general
---	---	---

Why the classes are separate, and not interchangeable

A pharmacy listing can support a pack size or a strength. It can never support “this works”, “at this dose” or “with this risk” — the citation guard reads the class off each source and cuts the sentence if no human study backs it. A journal domain is not enough either: NEJM and Nature publish mouse work constantly, so a journal page is peer-reviewed-other until the MeSH descriptor on the record itself establishes the species. Only the filtered PubMed path can do that.

Configuring PubMed

It works with nothing set. That is the first thing to know, because the previous arrangement did not look that way: the API key was read with `os.getenv` in the middle of the request builder, and the `pubmed_email` and `pubmed_tool` settings PAL already defined **were never sent to NCBI at all** — they existed and did nothing. One `PubmedConfig` now feeds every request.

```
$ python scripts/pubmed_setup.py          # what is in force, and where each value came from
$ python scripts/pubmed_setup.py --check  # the same, then one live request
$ python scripts/pubmed_setup.py --env    # lines to paste into api/.env
```

SETTING	DEFAULT	WHAT IT DOES
NCBI_API_KEY	unset	Raises the rate limit 3 → 10 req/s. Free from NCBI. Optional — everything works without it.
PUBMED_EMAIL	unset	Contact address sent on every request. NCBI asks for it so they can email you before blocking you. Set this one.
PUBMED_TOOL	PAL	How PAL identifies itself to NCBI.
PUBMED_MIN_YEAR	2000	Publication-year floor. See below.
PUBMED_MAX_RESULTS	8	Articles per query. Recall against synthesiser tokens.
PUBMED_TIMEOUT_SECONDS	12	A timeout reports “could not check the literature”, never “no evidence exists”.

Any of them can come from `api/.env`, from the environment, or from a call — `pubmed.configure(min_year=2010)`. `config.describe()` is safe to paste into a ticket: it says whether a key is set and how long it is, never what it is, so “missing key” stays distinguishable from “wrong key”. A typo'd value falls back to the default rather than failing the import, because a bad `.env` line should not take the API down.

NOTHING PUBLISHED BEFORE 2000

Old work is not wrong for being old. It is that it predates the trials, the guidelines and in many cases the drugs the patient asking is actually on — and a 1974 paper cited beside a 2023 one reads as equally current to someone with no way to tell.

Enforced twice, on purpose. Once as

`datetype=mdat&mindate=2000/01/01&maxdate=<this year>` on the search, and once on the records that come back, via `filter_by_year()`. Same reasoning as reading the Humans descriptor per record: the window is a *request*, the record is a *fact*. If NCBI ever changes its date parameters the first goes quiet and only the second still holds.

A record with no readable year is dropped too. Not being able to date a paper is not evidence that it is recent.

A relative window can tighten the floor and can never loosen it: `recent_years=200` still starts at 2000. One mechanism rather than two, because `reldate` and `mindate / maxdate` do not combine cleanly at NCBI.

Security findings

These are pre-existing and unrelated to the clinical work. They are first because two of them are more urgent than anything else in this document.

CRITICAL Live production secrets are in git history

`.env`, `.env.production` and `api/.env.production` are committed with populated values. Key names and value shapes confirm they are real, not placeholders:

`GEMINI_API_KEY` (AIzaSy..., 39 chars), `OPENAI_API_KEY`, `ANTHROPIC_API_KEY`, `POSTGRES_PASSWORD`, `CENTRIFUGO_TOKEN_HMAC_SECRET` (64 hex), and `SECRET_KEY` — the JWT signing key, which still carries a `change_...` prefix from a template.

`.gitignore` already listed `.env` and `.env.production`. The files were committed anyway, so the rules were bypassed rather than missing.

Required: rotate every key, then scrub history with `git filter-repo` or BFG. I have removed the files from the index and extended `.gitignore`, which stops the leak going forward — *it does not remove them from history*. Anyone with a clone still has them.

CRITICAL Patient data committed as a database dump

`backup_family_plan_20260812_145154.sql` (735 KB) contains `COPY` blocks for `patients`, `phone_users`, `health_facts`, `prescriptions`, `lab_tests`, `conversations` and `conversation_turns`, with 35 phone-number-shaped and 35 email-shaped strings. `pal_backup.sql` carries the same table set.

This is identifiable patient data in a source repository. Under the DPDP Act that is a personal-data breach if the repo has ever been accessible beyond the people entitled to that data — and the same history-scrub applies.

CRITICAL · FIXED Password authentication is broken on a fresh install

`requirements.txt` pinned `passlib[bcrypt]==1.7.4` but left `bcrypt` unpinned, so a fresh install resolves `bcrypt` 5.x. Reproduced:

```
bcrypt 5.0.0 → CryptContext.hash("...")
AttributeError: module 'bcrypt' has no attribute '__about__' (trapped)
ValueError: password cannot be longer than 72 bytes (fatal)

bcrypt 4.0.1 → hash OK, verify True
```

`passlib` 1.7.4 reads `bcrypt.__about__.__version__`, removed in `bcrypt` 4.1; `bcrypt` 5.x also stopped silently truncating at 72 bytes. Password hashing fails outright — not degraded, broken. **Fixed** by pinning `bcrypt==4.0.1`, verified working.

HIGH python-jose 3.3.0 signs and verifies every JWT

Used in `auth.py`, `auth_unified.py`, `auth_legacy.py`, `services/chat/principal.py` and `services/chat/authz.py`. Last released 2021; CVE-2024-33663 (algorithm confusion) and CVE-2024-33664 (JWE decompression DoS) apply to this version.

Mitigating: every `jwt.decode` call pins `algorithms=[settings.algorithm]`, which closes the classic confusion path. I checked all five. Migrating to `pyjwt` is still the right move; it is not an emergency. *CVE identifiers are from training knowledge — pip-audit would not install through this sandbox's proxy, so confirm against your own scan.*

REVIEWED · SAFE Two things that look like vulnerabilities and are not

`services/analytics/summary.py` builds SQL with an f-string, but the interpolated `group_col` comes from a two-branch if/else producing only `"doctor_id"` or `"source"` — never user input — and every other parameter is bound. Not injectable.

CORS in `api/main.py` is pinned to explicit localhost origins, not `*`.

Pre-existing defects, fixed

FIXED `synthesize()` could not run at all

Three stacked bugs meant the entire Hermes answer path was dead: the body referenced a `conversation_history` parameter that did not exist (`NameError`); the orchestrator passed that keyword anyway (`TypeError`); and the prompt template said `{conversation_history_section}` while `.format()` passed `conversation_history_section` (`KeyError`).

```
before: NameError: name 'conversation_history' is not defined
after : {'answer_text': 'ok', 'citations': []}
```

FIXED Conversation context was written but never read

`VectorizeHindsight.get_summary()` returned `""` unconditionally, with a comment deferring to the `pgvector` path. With `HINDSIGHT_ENABLED=true` the retain side wrote every turn to the conversation bank and the recall side read none of it back — so `conversation_history` reaching the synthesizer and the agents was always empty and the model saw each turn in isolation.

That is exactly the grouping problem the banked retention was built to solve. It now recalls the thread. **A second bug surfaced while testing it:** `_normalise_recall` slices `items[:top_k]` — the *oldest* entries — so a 60-turn thread returned turns 0–19 and dropped everything recent. Now keeps the tail.

FIXED `RoutingDepth.launch_hermes` did not exist

`test_fugu_complexity.py` asserts a `call`-complexity turn routes to it. The enum had `on_device | one | many | all` and `plan()` had no branch for `complexity == "call"` or `needs_action` — so "book a review with my doctor" fell through to the read-only paths and was answered as a question. That is the write path, and it was missing.

FIXED A test asserted that a missed stroke was correct

`test_stroke_symptoms_are_emergency` asserted `"my face is drooping and I can't speak" == "routine"`. The name said emergency; the assertion said routine. Corrected — and correcting it caught a live gap: `"his speech is slurred"` returned routine, because the pattern required `slurred` before `speech`. Both word orders now match.

Also: there were **two** safety implementations. The regex module `safety_triage.py` was never imported by the orchestrator, which was still on plain substring sets. Now wired, with a test asserting they agree.

ALSO FIXED	WAS
3 files did not parse	Quote-stripping damage: <code>"""</code> collapsed to <code>"</code> , literals unquoted, <code>\\$\\$</code> shell escaping in PL/pgSQL
<code>api/__init__.py</code> , <code>api/base.py</code>	Stray duplicates of the <code>models</code> package at the app root — made <code>api</code> a package, so every test import pulled in all of SQLAlchemy. Cause of the 43 collection errors. Removed; nothing referenced them.
<code>services/hermes/__init__.py</code>	Eagerly imported the orchestrator, so the "dependency-free" triage module needed a database driver. Now lazy (PEP 562).
<code>str(enum)</code> assertions	<code>str()</code> on a <code>(str, Enum)</code> member returns <code>"AgentName.evidence"</code> . Two tests could never pass. Now <code>.value</code> .
Root <code>DockerFile</code>	Unbuildable (no root <code>package.json</code>), unreferenced by either compose file, node:20, ran as root. Deleted. The live <code>web/Dockerfile</code> is node:24 and drops to the <code>node</code> user.
<code>test_identity_resolution.py</code>	Imports <code>IdentityResolver</code> , <code>IDValidator</code> and two more symbols that exist nowhere in the repo. Quotes restored so it parses; now skips with that stated.

The clinical layer

New package `api/services/clinical/`. Everything except `pubmed.py` is stdlib-only and importable without SQLAlchemy, a database or a network — the same property `safety_triage.py` relies on, for the same reason: rules that are awkward to run stop being run.

MODULE	WHAT IT DOES
<code>provenance.py</code>	Six source classes. Distinct from <code>evidence_class</code> : that says how well a claim is supported, this says what kind of source it is and therefore what it may support.
<code>pubmed.py</code>	E-utilities with <code>humans[MeSH Terms]</code> and real abstracts. Replaces a retriever that had no study filter and fetched summaries only. Includes a production canary.
<code>sources.py</code>	Journal/guideline/pharmacy allowlist. <code>site:</code> is a hint; the host filter is the guarantee — tested against <code>notbmj.com</code> and <code>bmj.com.evil.example</code> .
<code>drug_name_guard.py</code>	A medicine name is looked up only if it appears in the patient's own words. Infers and spell-corrects nothing.

MODULE	WHAT IT DOES
<code>citation_guard.py</code>	Post-generation check that a clinical claim cites a human study. Wired into the synthesizer with one repair pass.
<code>retention.py</code>	What may become durable memory. Refusals leave a content-free turn marker so the thread stays grouped.
<code>pharmacy_search.py</code>	Finds the product page for a brand the patient named. Two engines, two shapes, one brand-match rule that refuses substitutes.
<code>pharmacy_label.py</code>	1mg / PharmEasy label card with a mandatory disclaimer. Stays <code>commercial</code> provenance.
<code>drug_route.py</code>	The route end to end: patient's sentence → candidate names → guard → product page → card. Wired into the orchestrator.
<code>drug_resolver.py</code>	Brand → molecule behind an interface. Default resolves nothing rather than guessing.

THE TWO RULES WORTH HOLDING ONTO

A pharmacy page can be shown, never cited. Label info renders as an attributed card with a disclaimer; the citation guard still refuses it as support for a clinical claim in prose. A test fails if anyone later promotes pharmacy provenance.

A journal domain is not a human study. NEJM, Nature and BMJ map to `peer-reviewed-other`, not `peer-reviewed-human`. The domain establishes peer review; only the MeSH-filtered PubMed path establishes the species. Nature publishes mouse work constantly.

The drug route, now live

`pharmacy_label.py` could always turn a product page into a card. Nothing could find the product page — that was SearXNG's job, and SearXNG is not in PAL, so the module was unreachable code. `pharmacy_search.py` closes that, without adding a search service.

BOTH SITES WERE READ LIVE BEFORE ANYTHING WAS WRITTEN

They do not behave alike, and one of them behaves the opposite of the obvious guess.

SOURCE	WHAT IS ACTUALLY TRUE
PharmEasy	Search page <i>is</i> server-rendered — <code>/online-medicine-order/<slug></code> links are in the raw HTML. The product page carries <code>__NEXT_DATA__</code> with <code>compositions[]</code> , <code>drugStrengthValue/Unit</code> , <code>dosageForm</code> , <code>manufacturer</code> and <code>isRxRequired</code> as real fields. Parsed, not pattern-matched.
Tata 1mg	Search page is <i>not</i> server-rendered: <code>__INITIAL_STATE__</code> comes back with <code>searchResults: {}</code> and zero product links, so scraping the search HTML returns nothing at all. Its own JSON endpoint holds the results and rejects the request with <code>400 "City is required"</code> unless the city is sent as a header — passing it in the query string is not enough. The product page <i>is</i> server-rendered, with labelled fields.

PharmEasy is tried first because structured fields beat parsed prose; 1mg is the fallback and the broader catalogue.

SAFETY Both engines return substitutes, and a substitute must never reach the card

Searching `glycomet` returns **Biciphase** on 1mg and **Glidum MF** on PharmEasy. Different molecules, different manufacturers. Rendering one of those under the patient's own brand name is the brand substitution this entire route exists to refuse.

`brand_matches()` therefore returns **0 meaning reject**, not "rank last", on two conditions: the leading identity word differs, or the strength contradicts. The second matters as much as the first — *Glycomet 500* must not resolve to *Glycomet GP 1*, which is the same family with a second molecule in it and a different dose.

Two defects this work surfaced

FOUND	FIX
"Can I switch from Glycomet to Glucophage?" was not caught as a prescribing request. It passed every pattern in <code>PRESCRIBING_PATTERNS</code> . With the route live it would have been answered with two label cards side by side — a brand comparison in all but name.	Three substitution patterns added to <code>safety_triage.py</code> . Switch, swap, replace, substitute and "the same as" now exit at Stage 1b, to the doctor.
The extractor put CSS on the card. <code>_text_of</code> stripped tags but not <code><style></code> , so on the live 1mg page the manufacturer pattern matched a stylesheet rule and produced <code>Manufacturer-module__separator__P-NBE{width:100%...}</code> .	Scripts and styles removed first. Tags now flatten to <code> </code> rather than a space, which preserves field and list-item boundaries that a space destroys.

The one-way rule, enforced in four places

- **Names come only from the patient's sentence.** No step proposes one, so there is no path from a condition to a brand.
- **A word introduced by "for" is discarded.** "a medicine *for* hypertension" names an indication; it is never sent to a pharmacy.
- **`guard_names` still runs.** Redundant given where the candidates came from, and kept so it stays true when that changes.
- **Prescribing intent exits first.** Stage 1b is upstream of the route by construction, so comparisons never reach it.

Cost and failure

The lookup starts before the agent fan-out and is collected after synthesis, so its two HTTP requests overlap work already happening rather than adding to it. A turn that is not about a named medicine costs one regex. Every failure — dead network, redesigned page, unlisted brand, slow pharmacy, exceeded budget — returns an empty list, and the turn is answered exactly as it was before the route existed.

`drug_label_lookup_enabled: false` switches the whole thing off.

Perplexica is not integrated

Stated plainly because it is a real gap: there are **zero** Perplexica references in PAL — not in the code, not in `docker-compose.yml`, not in requirements. What shipped is a Python port of the clinical *rules*, not Perplexica.

What this still costs: `journal_site_query()` and `is_allowed_journal_url()` are defined and tested but unreachable at runtime, because nothing in PAL can search the open web. NEJM, BMJ, Nature, Medscape and bioRxiv are allowlist entries with nothing to apply them to.

What it no longer costs: the drug card. That was the second dead module and it is now live end to end, because each pharmacy is reached through its own site rather than through a general search engine. Two of the three routes — PubMed and drugs — are therefore direct API calls and need no Perplexica at all.

Smallest unblocking step for the third: SearXNG as a compose service plus a ~60-line client activates the journal allowlist. Porting the researcher loop (~600 lines) is the actual merge and can follow.

Running and testing

```
# tests – no network, no database
cd api && python -m pytest tests/ --ignore=tests/integration -q # 250 passing

# PubMed setup: what is configured, from where, and one live request
python scripts/pubmed_setup.py --check

# proves humans[MeSH Terms] is valid against the live index
python scripts/verify_pubmed_live.py # exit 0 = all six checks held

# proves the drug route still works against the live pharmacies.
# Run this when a card looks wrong or empty – fixtures cannot notice a redesign.
python scripts/verify_pharmacy_live.py # or: ... "Dolo 650"

# clinician review artefact
python scripts/review_claim_cues.py # markdown table
python scripts/review_claim_cues.py --measure # precision/recall

# optional, raises the PubMed rate limit 3/s → 10/s
export NCBI_API_KEY=...
```

The filter is now verified against the live index (see the first section), and guarded three ways: `verify_pubmed_live.py` on demand, `filter_sanity_check()` once on first use in production, and per-record MeSH verification on every single article. The first two treat *unreachable* as different from *broken*; the third makes a broken filter downgrade provenance rather than mislabel animal work as human evidence.

Adversarial audit, and what it found

The new code was put through a deliberate attempt to break it, and then a patient conversation was role-played through it end to end. Every claim below was reproduced by running it, not by reading. Ten defects, three of them in properties this document had already asserted. All are fixed, each with a test named after the defect rather than the function, in `tests/test_audit_regressions.py`.

WAS A substitute could wear the patient's brand name

`brand_matches` checked only the *first* word. `Glycomet` scored identically against `Glycomet 500` and `Glycomet GP 1` — and the tie was broken by the pharmacy's own relevance ranking, the very thing the module exists to distrust. 1mg ranks GP first.

Glycomet is metformin. Glycomet GP adds glimepiride, a sulfonylurea that causes hypoglycaemia. Same shape for Ecosprin 75 against Ecosprin AV (aspirin, vs aspirin + atorvastatin) and Pan 40 against Pan D. A patient reading a card headed with their own brand would have been reading the label of a different drug.

Now: the match is symmetric — every identity token the patient typed must be present, and none they did not. Release markers (SR, XR, PR) are the one exception, penalised rather than refused, because they are the same molecule in a different formulation. Fourteen cases pinned in tests.

WAS The year on a citation was not the publication year

`_parse_articles` took the first `<Year>` in the record. In the NLM DTD that is `DateCompleted`, or `DateRevised` on an ahead-of-print record — an indexing date, never publication.

A 1997 paper with DateRevised 2021 read as 2021. It passed the year floor, was shown to the patient dated 2021, and `filter_by_year` — the check that exists precisely to catch a broken date window — waved it through, because it was reading the wrong element. On ordinary records it was merely wrong by a year or two, which is how it survived: **7 of 20 records in the live run above had the wrong year.**

Now: `PubDate`, then `ArticleDate`, then `MedlineDate` for ranges like “2003 Jan-Feb”. No readable publication date means the article is dropped.

WAS The fetcher could be walked off the allowlist

`follow_redirects=True` validates nothing — httpx walks the chain internally, so checking the final URL meant every intermediate hop had *already been requested*. An open redirect on a pharmacy host, and the starting URL comes from a pharmacy's own search results, was enough to make PAL issue a GET against any address reachable from our network. Reproduced against `169.254.169.254`: the request went out and the call returned normally.

Now: no automatic redirects. Each hop is checked before it is requested, and the test asserts the internal address is never contacted. `is_pharmacy_url` also checks the scheme, so `file://img.com/etc/passwd` no longer satisfies it.

WAS One page could stall every other request in the process

`<(script|style)\b[^>]*>. *?</\1>` rescans to end-of-document from every unclosed opening tag. Not backtracking — plain $O(n \cdot m)$ — and just as bad:

8,000 stray <script>	in 257 KB	11.4 s
16,000 stray <script>	in 320 KB	25.8 s
100,000 stray <script>	in 4 MB	did not finish in 90 s

Synchronous work on the event loop, so the 12-second budget could not interrupt it and every concurrent request waited behind it. Measured: an 8.3 s stall of an unrelated task on a 1 s budget.

Now: a single left-to-right scan, no regex. The 16,000-tag page takes under a millisecond, and an unclosed tag drops the remainder rather than parsing it.

WAS Four ways to print wrong text on a patient's card

INPUT	PRODUCED
"Extra 15% off" above the composition	strength 15%
A "similar products" link to Metformin 500mg, on the page for Glycomet 850	strength 500mg beside a composition line reading 850mg — the card contradicting itself, with no way for the patient to tell which was right
"The manufacturer and the marketplace are not liable..."	manufacturer and the marketplace are not liable...
"...to avoid decomposition of the active ingredient"	composition of the active ingredient
"The misuses of this medicine include ... and dependence"	uses ['ependence'] — a mid-word fragment, from a sentence that was not a uses section

Now: strength is read out of the composition, which is the field that is actually about this product, falling back to the page only within sight of the brand name. Field labels require their separator, so prose cannot pose as one. Headings are word-bounded. A missing field is recoverable; a field confidently filled with small print is not.

WAS The guard refused the drug route's own output

Found by role-playing a patient conversation through the shipped code, which turns out to be a different exercise from reading it. The guard rejected the one thing the drug route exists to produce:

```
"Glycomet 500 contains metformin hydrochloride 500mg[1]." -> REFUSED
```

The dose-units cue fired on 500mg, and there is no human study behind a composition, because a composition is not a finding — it is what is printed on the strip. Left alone, the synthesiser could never state the contents of the card sitting beside it: it would strip the sentence, or loop on the repair pass, on every single medicine question. **The display-versus-cite distinction the whole design rests on would have collapsed the first time a patient asked about a medicine**, and no test caught it because every existing test asserted what the guard *refuses*.

Now: a narrow exemption. A sentence qualifies only if the units cue is the *only* claim cue it triggers, it says one of the label things (contains, composition, each tablet, marketed by, pack of, prescription-only), and it carries no instruction language. Take 500mg twice daily, contains 500mg and reduces HbA1c by 1%, contains 500mg and is safe in pregnancy are all still refused — and the exemption relaxes *which* source may support a label statement, never whether one is needed.

ALSO FIXED Four smaller ones

- **A condition reached the pharmacy.** “for the treatment of hypertension” slipped the indication guard — its lookback was 14 characters and the phrase is 21 — so `hypertension` went out in a URL to a commercial third party. Both an inversion of the one-way rule and a privacy leak.
- **Greetings ate the lookup budget.** “Hello sir, tell me about Glycomet 500” spent both candidate slots on *Hello* and *sir*; the medicine was never looked up. A silent miss, and a likely one in the target cohort. Candidates are now ranked by whether the sentence points at them.
- **NCBI can fail with a 200.** A holding page raised `JSONDecodeError` out of `search_pubmed_many`, which catches only httpx errors — breaking the turn. A rejected term came back `200 OK` with an `ERROR` field and no `idlist`, reported as `retrieval_ok=True`, i.e. “no evidence exists”. Both now produce `retrieval_ok=False`.
- **Devanagari digits scored zero.** Python’s `\d` is Unicode-aware, so “Glycomet ५००” shared no digit run with “500” — no card, for a correctly named medicine, in exactly the languages this is for.

What the audit did not find, having tried: no allowlist bypass through IDN, userinfo, case, trailing dot or homograph hosts; no catastrophic backtracking; and no path that promotes a pharmacy source above `commercial` provenance — the display-not-cite rule held everywhere it was attacked.

Deploying

Nothing new has to be configured. Every setting added here has a working default, and `.env.example` now documents all nine with what each one does — so a developer reads one file rather than the source.

```
cp .env.example .env          # the PubMed and drug-route block is at the end
docker compose up -d --build

# confirm what the container actually picked up
docker compose exec api python scripts/pubmed_setup.py --check
docker compose exec api python scripts/verify_pharmacy_live.py
docker compose exec api python scripts/walkthrough.py
```

Both compose files already pass the whole env file through (`env_file: .env / .env.production`), so no compose change is needed — a variable set in the env file reaches the API container as it is.

VARIABLE	DEFAULT	SET IT WHEN
<code>PUBMED_EMAIL</code>	unset	Always, for a deployment serving real patients. NCBI emails you before blocking you.

VARIABLE	DEFAULT	SET IT WHEN
NCBI_API_KEY	unset	Traffic outgrows 3 requests/second. Free.
PUBMED_MIN_YEAR	2000	You want a different cut-off.
PUBMED_MAX_RESULTS	8	Trading recall against synthesiser tokens.
PUBMED_TOOL / PUBMED_TIMEOUT_SECONDS	PAL / 12	Rarely.
DRUG_LABEL_LOOKUP_ENABLED	true	You want the drug cards off. Turns are then answered exactly as before the route existed.
DRUG_LABEL_CITY	Gurgaon	You want 1mg serviceability for a different city. Affects which listings are shown, never the label.
DRUG_LABEL_TIMEOUT_SECONDS	12	Rarely. Exceeding it drops the cards, never the answer.

SCRIPTS/WALKTHROUGH.PY

Replays an eleven-turn patient conversation — diagnosis question, three brand names, three refusals, a condition that must not be looked up, Hindi, and an emergency — through the shipped functions, printing which stage fired at each turn.

--live swaps the recorded pharmacy pages for the real ones.

It is the fastest way to see that the ordering holds, and it is how the tenth defect above was found. The recorded pages deliberately contain the substitutes both engines really return (Glidum MF and Glycomet GP for “glycomet”, Pan D for “Pan 40”), so if the brand-match rule ever loosens, the walkthrough shows it on the next run.

Still open

ITEM	BLOCKED ON
Rotate the leaked keys, scrub git history	Nothing. Do this first.
Clinician sign-off on the claim cues	A clinician; the artefact exists
SearXNG service	A decision. Unblocks two dead modules.
Drug data source	Legal read on scraping 1mg/PharmEasy
PHI boundary at synthesis	context_contains_phi() is the hook; synthesis does not yet pick its model from it
Integration tests	Live Postgres — and the resolver they target does not exist yet
python-jose → pyjwt	Nothing, but not urgent given the pinned algorithms

Nothing was pushed. The zip is a working tree with the patch applied; CLINICAL-LAYER.patch inside it is the diff against 60485bb. The branch and the commit are yours to make.

The zip deliberately excludes `.git`, all `.env` files and the database dumps — shipping them would spread the exposure described above. None of them is source code; the tree is complete without them.